Leela BT3 INT8 roadmap (4 weeks, final)

Ritwik Mishra (ME23B237) and Anjan R D (ME23B157), CS6886 Systems for Deep Learning, IIT Madras

We quantize the BT3 transformer from Leela Chess Zero to INT8 with TensorRT and run the quantized network inside lc0 itself. An ablation measures what each compression technique adds, including SmoothQuant and 2:4 structured sparsity. Every model is compared with FP16 at three levels: the network on its own, search with a fixed number of nodes, and search with a fixed amount of time. Three hardware studies explain the speed results: a per-layer profile with a roofline, energy per node, and search batch size. The main output is one plot of tablebase accuracy against nodes/s with every model on it.

Research questions

1. How much does INT8 change BT3's policy and value outputs, and which compression techniques reduce that change?
2. With a fixed node budget, how much of the damage does search hide? Is the search more sensitive to errors in the policy or in the value?
3. With a fixed wall-clock budget (and a fixed energy budget), does faster inference buy enough extra search to make up for the lost quality?

Fixed decisions

Decision	What we fixed
Network	One BT3 checkpoint, exported with lc0's `leela2onnx`
Precision	INT8 only (W8A8), compared with an FP16 baseline
Quantization	QDQ ONNX produced with NVIDIA ModelOpt. PyTorch (from `leela2onnx --onnx2pytorch` or a manual port) is used only for QAT and KD

Harness	lc0 v0.33 with the `onnx-trt` backend and `optimize=8`. Our ONNX is wrapped into an lc0 network file with `onnx2leela`. ONNX Runtime stays at 1.26 or earlier, because lc0 warns that 1.27+ silently computes in FP32. Checked on day 1. If it fails, timed runs use a batched Python MCTS
Swap study	A small Python MCTS, used only at fixed node counts, so its speed doesn't matter
Test set	Syzygy tablebase positions, plus a hard subset where FP16 is close to getting the move wrong
Scoring	A move is correct if it keeps the tablebase win/draw/loss result, as in Haque et al. The DTZ-optimal rate is reported as a secondary number
Protocol	lc0 gets no tablebase path during search. For timed runs the GPU clock is locked with `nvidia-smi -lgc` if the laptop allows it, the laptop is plugged in with a fixed power mode, temperature is logged, every run is warmed up and repeated, medians are reported, and only one engine uses the GPU at a time
Reference	Stock lc0 native CUDA FP16 nodes/s, recorded as a check on our FP16 baseline
Out of scope	FP8, W4, custom kernels, middlegame test sets, the clock-scaling study, alpha-beta search, other networks

Scope tiers

Tier	Contents
Must	Ablation steps 1 to 5 (SmoothQuant included), 2:4 speed ceiling (S0), Studies A, B, B2 and C, energy, batch size, `trtexec` profile, the frontier plot

Should	QAT and KD (steps 6 and 7), SparseGPT 2:4 (S1), the per-layer roofline, the best sparse model in the timed studies
Could	Sparse recovery with KD (S2, only if QAT is running)

Team split

Person	Owns
Person A (model)	Export, the quantization ladder, SmoothQuant, SparseGPT, QAT and KD, NN-level metrics, the policy/value swap study, and the roofline if QAT is dropped
Person B (systems)	lc0 and TensorRT setup, scoring, fixed-node and fixed-time runs, matches, the 2:4 speed ceiling, profiling, energy, the batch sweep

After Week 1, neither person changes the model format, precision definitions, test set or metrics without the other agreeing.

Week 1: FP16 pipeline end to end

Step	Person A	Person B
1. Freeze the spec	Pick the checkpoint and define every variant with B.	Node budgets, batch settings, scoring rules.
2. Export	`leela2onnx` to ONNX, and `--onnx2pytorch` for the PyTorch model. List every matmul and its shape.	Build lc0 v0.33 with `onnx-trt`. Install TensorRT and `trtexec`. Record stock lc0 FP16 nodes/s.

3. Harness test (days 1 and 2)	A quick naive INT8 QDQ ONNX from ModelOpt, wrapped with `onnx2leela`.	Run both files in lc0 with `optimize=8`. Check the logs for nodes that fall back from TensorRT to the CUDA or CPU providers. If it fails, start the Python MCTS fallback straight away.
4. Data	Calibration, validation and test splits from the tablebase positions, with no overlap.	Syzygy scoring (WDL, DTZ secondary) and the hard subset.
5. Outlier check	Plot the largest value in each activation channel on calibration data, to see where SmoothQuant will matter.	Check whether `nvidia-smi -lgc` works on the laptop.
6. Baselines	FP16 NN-level metrics.	FP16 fixed-node sweep, nodes/s, `backendbench` throughput by batch size, `trtexec --dumpProfile --dumpLayerInfo`, and the GPU (NVML) and CPU (RAPL) energy scripts.

Gate C1: FP16 runs end to end in lc0, with baseline accuracy, nodes/s, profile and energy numbers.

Week 2: INT8 ladder and the sparsity ceiling

Step	Person A	Person B

7. Ablation steps 1 to 5	Naive PTQ, then per-channel weights, then entropy or percentile calibration, then SmoothQuant, then calibration on positions the search actually visits.	An engine for each step. Use `--dumpLayerInfo` to confirm the matmuls run in INT8, and fix QDQ placement if they don't.
8. Studies A and B	NN-level metrics for every step.	Fixed-node sweep for every step.
9. 2:4 speed ceiling (S0)	Nothing.	Magnitude 2:4 pruning on the FP16 weights, then INT8 with the best recipe so far, then an engine built with `--sparsity=enable`. Confirm the layer info shows sparse kernels and measure the speedup.
10. QAT decision (mid-week)	Go ahead if the PyTorch model matches lc0's outputs and training fits on some GPU. Otherwise drop steps 6 and 7 and S2.	Nothing.

Gate C2: the best PTQ model is frozen and the 2:4 speed ceiling is measured. This is the safety net. From here the project is complete even if nothing else works.

Week 3: training, sparsity, timed and hardware studies

Step	Person A	Person B
11. QAT and KD (if kept)	Steps 6 and 7, starting from the best PTQ recipe. If S2 is on, apply the 2:4 mask in the same training run.	Engines for the trained models.

12. SparseGPT (S1)	One-shot 2:4 pruning with Hessian compensation on the FP16 weights (an extension of the Assignment 2 GPTQ code), then INT8 with the best recipe.	Build and benchmark.
13. Policy/value swap (B2)	Small fixed-node Python MCTS that takes the policy from one network and the value from the other. Run INT8 policy with FP16 value, and FP16 policy with INT8 value.	Nothing.
14. Study C	Nothing.	Fixed-time accuracy for FP16, best INT8 and best 2:4 INT8.
15. Matches (overnight)	Pick the openings with B.	About 1000 games of best INT8 against FP16 at 100 ms per move, colors swapped. Add best 2:4 against FP16 if there is time.
16. Energy	Nothing.	Joules per node (GPU and CPU, above idle) and accuracy at equal energy budgets, for FP16, best INT8 and best 2:4.
17. Batch size	Nothing.	Minibatch sizes 8 to 512: nodes/s, fixed-node accuracy and fixed-time accuracy for FP16, best INT8 and best 2:4.

18. Roofline	FLOPs and bytes for each matmul from the shapes, at one small and one large batch size.	Match `trtexec` layer timings to the matmuls (capped at one day) and add the 4060's peak compute and bandwidth at the locked clock.

Gate C3: all results frozen. No new models or experiments after this point.

Week 4: analysis and report

Period	Person A	Person B
Early	Ablation, node-equivalents, policy/value swap and outlier plots.	Fixed-time, match, profile, roofline, energy and batch-size plots, and the frontier plot.
Mid	Write the background, quantization method, ablation, NN-level and fixed-node results, and the policy/value finding.	Write the harness, TensorRT pipeline, benchmarking protocol, timed results and hardware studies.
Late	Together: abstract, introduction, related work (Haque et al., GPTQ, SmoothQuant, SparseGPT, Stockfish NNUE), discussion, limitations, conclusion. Clean the code and run a reproduction test from a fresh clone.	Check every number in the report against its log. Verify the Haque et al. figures and all citations.

Gate C4: report, code and presentation frozen.

Compression ablation

The main ladder is cumulative. Each step keeps everything before it and adds one technique.

Step	Model	What it adds
0	FP16	Baseline
1	Naive W8A8 PTQ	Per-tensor weights, max calibration, random positions
2	+ per-channel weights	A separate scale for each output channel
3	+ better calibration	Entropy or percentile instead of max
4	+ SmoothQuant	Moves activation outliers into the weights, with the scales folded into the preceding layernorm
5	+ search-aware calibration	Calibration on positions the search actually visits
6	+ QAT	Fine-tuning with fake quantization (Should)
7	+ KD	Distillation from FP16 during QAT (Should)

The sparse branch prunes the FP16 weights first. It then quantizes the pruned network with the best recipe from the ladder and builds a sparse INT8 engine. It sits later in the schedule only because it needs the ladder's result.

Step	Model	What it tests
S0	Magnitude 2:4, then INT8	The speed ceiling of the sparse tensor cores. It drops the two smallest weights in every group of four and uses no data
S1	SparseGPT 2:4, then INT8	Accuracy without training. It drops the weights whose removal changes the layer output least and adjusts the remaining weights to compensate (Should)

S2	SparseGPT 2:4, then INT8 with KD	Accuracy with recovery training (Could)

Steps 1 to 7 all run at the same speed, so they form a quality study (Studies A and B only). The sparse models run at a different speed, so the best one also goes into the timed studies.

Comparison studies

Study	Models	Question	Measurements
A. Network only	All ablation steps	How much does each model change the network?	Policy KL against FP16, top-move agreement, value/WDL error, raw-policy tablebase accuracy
B. Fixed nodes	All ablation steps	How much of the damage does search hide?	Tablebase accuracy at 0, 100, 400, 800 and 1600 nodes, and node-equivalents
B2. Policy/value swap	FP16 and best INT8	Which output's error hurts the search more?	Fixed-node accuracy of both swaps, against FP16 and full INT8
C. Fixed time	FP16, best INT8, best 2:4	Does the speed pay off?	nodes/s, tablebase accuracy at equal time, matches (Elo with a confidence interval)

Hardware studies

Study	What it measures	Why it's in the project
Profile and roofline	Time per layer from `trtexec`, and each matmul's achieved throughput against the 4060's compute and memory limits	Explains which layers INT8 speeds up and which ones set the limit

Energy	Joules per node on GPU and CPU above idle, and accuracy at equal energy budgets	Tests whether INT8 is stronger per joule as well as per second
Batch size	nodes/s and accuracy for minibatch sizes 8 to 512	Bigger batches use the GPU better but make the search less exact. The best batch size may differ between FP16 and INT8

Final deliverables

Code repository. Export, quantization ladder, SmoothQuant, sparsity, training, the swap MCTS, scoring, benchmarks, and the energy and batch scripts. One script per experiment, and a README that reproduces FP16, best PTQ and best 2:4 from a fresh clone.

Model artifacts. ONNX files, lc0 network files, TensorRT engine caches, configs and calibration data for every model.

Test set. Tablebase position lists, the hard subset, the splits and the scoring scripts.

Tables.

- T1: ablation (all steps, Studies A and B)
- T2: speed (latency, NN evals/s, nodes/s, INT8 and sparse layer coverage, stock lc0 reference)
- T3: energy (joules per node, GPU and CPU)
- T4: matches (Elo with a confidence interval)
- T5: 2:4 speed ceiling (per layer and end to end)

Figures.

- F1: ablation ladder
- F2: accuracy against node budget
- F3: node-equivalents
- F4: policy/value swap
- F5: accuracy against time budget
- F6: per-layer time breakdown and roofline
- F7: joules per node
- F8: accuracy at equal energy
- F9: nodes/s against batch size

- F10: fixed-time accuracy against batch size
- F11: accuracy against nodes/s with every model on it (the headline plot)
- F12: activation outlier profile before and after SmoothQuant

Report and presentation, with a short demo of the INT8 engine playing in lc0 with its nodes/s on screen.

Risks and fallbacks

Risk	When we'll know	Fallback
lc0's `onnx-trt` won't run our INT8 file, or splits the graph badly	Day 2	Batched Python MCTS for timed runs, logging the network's share of wall time
Matmuls stay in FP16 inside TensorRT	Week 2 layer check	Fix QDQ placement before any evaluation
BT3 has no real activation outliers	Week 1 outlier check	Keep step 4 in the ablation and report that it changes little
TensorRT doesn't pick sparse kernels	Week 2, S0	Report that as the finding and drop S1 and S2
SparseGPT loses too much accuracy	Week 3, S1	Report S0 as a speed-ceiling result only
PyTorch model is late or QAT doesn't fit in memory	Mid-Week 2	PTQ only. Drop steps 6 and 7 and S2
INT8 is barely faster than FP16	Week 2	Explain it with the profile, roofline and batch study. It is still the answer to RQ3
Tablebase errors are too rare to separate models	Week 1 baseline	Enlarge the hard subset

Clock locking isn't allowed on the laptop	Week 1	Rely on warm-up, temperature logs and medians

Limitations to state in the report

One network on one GPU. Per-move accuracy comes from endgames only, so the matches are the only evidence from full games. The swap study uses a simpler search than lc0. FP8 and the clock-scaling study were left out on purpose.

Open questions for Week 1

- Which exact BT3 checkpoint?
- Is a lab or cluster GPU available for QAT?

Afternote (optional, not part of the proposal): partial INT8. Keep the one or two body layers that are most sensitive to quantization in FP16, chosen with a per-layer sensitivity sweep, and quantize everything else. Only worth doing if best INT8 PTQ shows a clear accuracy drop against FP16 on the hard tablebase subset and QAT has been dropped. If it goes ahead, it adds one point to the frontier plot and joins Study C.